SpotSync

Real-Time Parking Discovery, Reservation & Contactless Ticketing Application

System Documentation, Architectural Specification & Viva Demonstration Guide

SRS Version 1.1 (Revised) • RMMM Plan Compliance

Department of Computer Engineering • Sardar Patel Institute of Technology (SPIT)

Prepared by: Shrey Vadhani & Maisha Vageriya

1. Executive Summary & Project Purpose

Urban traffic congestion is heavily exacerbated by motorists searching for available parking spaces, leading to lost time, fuel wastage, and increased urban carbon emissions. **SpotSync** is a full-stack, real-time parking discovery, advance reservation, and contactless ticketing platform designed to resolve urban parking friction for city centers, office hubs, and campus environments.

Built precisely according to the IEEE Std 830-1998 Software Requirements Specification (SRS v1.1), the application incorporates real-time WebSocket availability broadcast (<5 seconds propagation), transactional locking to eliminate double-booking, turn-by-turn GPS navigation with live ETA, cryptographically tokenized QR passes for contactless entry/exit gates, and automated fee computation based on actual stay duration.

2. System Features & SRS Requirement Traceability

The system encompasses six core functional feature groups mapped directly to SRS Section 4:

4.1 Search Parking Spaces by Location (REQ-1 to REQ-6)

- **Automatic Location Detection (REQ-1):** Automatically senses user coordinates using the Geolocation API, centering the search on current position with default fallback to Sardar Patel Institute of Technology campus (`19.1232, 72.8360`).
- **Landmark & Destination Search (REQ-2):** Text search allowing drivers to find lots near destinations (e.g., SPIT, BKC, Andheri, Lower Parel).
- **Interactive Leaflet Map & Card Views (REQ-3, REQ-4):** Visual pins on OpenStreetMap indicating parking locations with real-time slot counters, paired with a toggleable list view.
- **Sorting & Radius Filtering (REQ-5):** Configurable radius (1 km to 20 km) with instant sorting by distance (Haversine formula), hourly fee, or open slot count.
- **Auto-Radius Expansion (REQ-6):** System automatically notifies the driver and expands the radius when no facilities fall within initial parameters.

4.2 Real-Time Parking Availability (REQ-7 to REQ-11)

- **Live Slot Counter (REQ-7):** Accurately reports open spots against total capacity per facility.
- **WebSocket Propagation <5s (REQ-8):** Changes in slot occupancy are pushed via bi-directional WebSockets to all connected client browsers in real time without polling delay.
- **Visual Color-Coded Indicators (REQ-9):** Color-coded badges categorizing lots: `AVAILABLE` (healthy capacity), `FILLING FAST` (<30% remaining), and `FULL` (0 slots).
- **Double-Booking Concurrency Lock (REQ-10):** Database-level atomic transactions lock slot state during booking, resolving Risk R1 from the RMMM Plan.
- **Admin Maintenance Mode (REQ-11):** Parking Lot Administrators can manually mark slots for maintenance or repair, instantly pulling them from public availability.

4.3 Slot Reservation & Cancellation Lifecycle (REQ-12 to REQ-17)

- **Specific Slot Selection (REQ-12, REQ-13):** Interactive graphical matrix displaying individual spots (Standard 🚗, EV Charging ⚡, Accessible ♿, Compact 🏎️) for custom time windows.
- **Reservation History (REQ-14):** Centralized log displaying active, checked-in, completed, and cancelled tickets.
- **Cancellation & Immediate Release (REQ-15, REQ-16):** Drivers can cancel reservations prior to arrival, automatically returning the slot to the open pool.
- **15-Minute Grace Period Background Monitor (REQ-17):** Automated background worker detects unclaimed reservations 15 minutes past start time, cancels the pass, issues a notification, and frees the spot.

4.4 GPS Navigation & Routing (REQ-18 to REQ-21)

- **Turn-by-Turn Route Preview (REQ-18, REQ-19):** Interactive polyline navigation overlay rendered directly inside the application.
- **Dynamic ETA & Distance (REQ-20):** Displays remaining travel distance in kilometers and estimated arrival time based on urban transit speeds.
- **External Maps Integration (REQ-21):** Direct link launching native Google Maps navigation with destination coordinates.

4.5 Contactless Digital QR Code Ticketing (REQ-22 to REQ-26)

- **Signed QR Generation (REQ-22):** Cryptographically signs a unique token using HMAC-SHA256 immediately upon booking confirmation (`SPOTSYNC:res_id:SIGNATURE`).
- **Offline Caching (REQ-23):** Passes are cached in browser `localStorage`, allowing drivers to display their ticket even when cellular reception is lost.
- **Gate Entry Validation (REQ-24):** Lot admin scans QR pass at entrance gate, verifying token validity, marking ticket as `entered`, and recording exact arrival timestamp.
- **Gate Exit Scan & Automated Billing (REQ-25):** Admin scans QR code at departure, records exit timestamp, calculates duration and billable hours, computes total fee, marks pass `exited`, and returns the slot to available pool.
- **Fraud & Replay Prevention (REQ-26):** Explicit rejection of forged, expired, or already-consumed passes.

4.6 Real-Time Notifications (REQ-27 to REQ-30)

- In-app alert banners and persistent notification drawer logging booking confirmations, 30-minute arrival reminders, and cancellations.
- Profile preference toggles allowing drivers to configure which notification categories they receive (REQ-30).

3. Technical Architecture & Implementation Details

> **Architecture Philosophy**
>
> SpotSync implements a modular, decoupled full-stack architecture prioritizing sub-second latency, zero hardcoded logic, strict ACID data integrity, and cross-platform responsive UX.

Layer	Technologies Used	Key Responsibilities
Backend API	Python 3.12, FastAPI, Uvicorn, Starlette	RESTful endpoints, request validation, business logic, CORS, static file serving.
Real-Time Stream	Native WebSockets (`/ws/lots`)	Broadcasts slot state changes across all connected clients in < 5 seconds (REQ-8).
Database	SQLite (WAL Mode, Foreign Keys ON)	ACID transactions, relational schema (Users, Lots, Slots, Reservations, Tickets, Notifications).
Security	PyJWT, HMAC-SHA256, SHA-256	Role-Based Access Control (RBAC), signed digital QR passes, password hashing.
Background Worker	Asyncio Event Loop Task	Periodically scans for grace period expiries (15 mins) and auto-releases abandoned slots.
Frontend Client	HTML5, Tailwind CSS, Lucide Icons	Modern glassmorphism mobile-first SPA, interactive slot matrix, notification drawer.
Mapping & GPS	Leaflet.js & OpenStreetMap	Interactive pins, custom color-coded badges, routing polyline, distance calculation.
QR Engine	QRCode.js (Client) + HMAC (Backend)	Vector QR rendering, offline storage, camera/manual validation gate portal.

Data Model & Referential Integrity (Appendix B.2)

- `users` : Primary key `user_id` , role-based classification (`driver` , `lot_admin` , `sys_admin` , `guest`), notification settings.
- `parking_lots` : Stores facility name, address, latitude/longitude coordinates, pricing per hour, and total slots.
- `slots` : Relational child of `parking_lots` . Tracks `slot_number` , vehicle type (standard, ev, accessible, compact), and live status (`available` , `booked` , `maintenance`).
- `reservations` : Relational bridge linking user, slot, lot, scheduled start/end windows, and booking status (`active` , `checked_in` , `completed` , `cancelled` , `expired`).
- `tickets` : 1-to-1 relationship with reservations. Contains signed QR code, entry timestamp, exit timestamp, fee amount, and gate status.
- `notifications` : User-specific alert history with category tagging and read/unread flags.

4. Live Viva & Project Demonstration Guide

Follow this recommended step-by-step presentation script during your project viva or evaluation to demonstrate 100% of the functional and non-functional requirements:

Phase 1: Starting the System & Proving Zero Hardcoded Logic

1. Open VS Code and run `python backend/app.py` (or double click `start.bat`).

2. Point the evaluators to `http://localhost:8000/docs` to show the automated Swagger/OpenAPI documentation, demonstrating full REST API connectivity.

3. Navigate to `http://localhost:8000/` in the browser to display the responsive, modern Glassmorphism UI.

Phase 2: Location-Based Search & Interactive Map (Driver Mode)

1. Select **Driver Mode** using the role switcher pills at the top navbar.

2. Click **"My Location"**: Show how GPS coordinates dynamically resolve and center the map on the college campus (SPIT Mumbai).

3. Test the search bar: Type `BKC` or `Andheri` to demonstrate real-time landmark filtering.

4. Adjust the **Radius Slider** from 5 km to 10 km: Point out the calculated distance in kilometers and the color-coded status badges (`AVAILABLE`, `FILLING FAST`, `FULL`).

5. Toggle between **Split View**, **Map Only**, and **Card View** to show UI flexibility.

Phase 3: Visual Slot Grid & Transactional Reservation

1. Click **"Reserve Slot"** on *Sardar Patel Inst. of Technology Parking*.

2. Show the **Visual Matrix**: Explain how slots are categorized by type (🚗 Standard, ⚡ EV Charging, ♿ Accessible, 🏎️ Compact).

3. Click an available slot (e.g., `A-01`), select duration (e.g., 2 Hours), and review the live price calculation.

4. Click **"Confirm & Generate Ticket"**: Emphasize that the reservation was executed via an atomic database transaction (REQ-10) to prevent double-booking.

Phase 4: Proving Double-Booking Prevention (RMMM Risk R1)

1. Open a second browser window (or Incognito mode) side-by-side.

2. Show that slot `A-01` immediately updated to **BOOKED** on the second client without refreshing the page (demonstrating WebSocket sync under 5 seconds).

3. Attempting to reserve the same slot returns a clean conflict rejection error, proving concurrency safety.

Phase 5: Digital QR Pass & Offline Caching

1. In the first window, navigate to **"My Bookings"**.

2. Click **"View Digital QR Ticket"**: Show the generated HD QR code and explain that it is cryptographically signed with HMAC-SHA256.

3. Highlight that the ticket is stored in `localStorage`, satisfying REQ-23 (offline availability).

Phase 6: Gate Attendant Validation (Entry & Exit Simulation)

1. Switch to **Lot Admin** mode via the top navbar.

2. Open the **"Gate Scanner"** tab.

3. Click **"Auto-Fill"** (or paste the ticket code) and click **"Validate ENTRY Gate Scan"**:

- Show the green confirmation banner and the exact entry timestamp recorded in the database.

4. Click **"Validate EXIT Gate Scan"**:

- Show the departure timestamp, computed parking duration, and total fee calculated based on the hourly rate.
- Show that the slot was automatically released back into the available pool for future drivers.

Phase 7: System Administration & Grace Period Monitor

1. Switch to **Sys Admin** mode and open the **"Admin Dashboard"**.

2. Display the live analytics cards: Total Lots, Total Slots, Available Spots, and Total Revenue Collected.

3. Click **"Onboard New Parking Lot"**: Fill in sample coordinates and submit, demonstrating dynamic lot provisioning.

4. Explain the **15-Minute Grace Period Background Task**: Highlight that background tasks continuously protect parking operators against no-shows.

5. Key Viva Questions & Model Answers

Question	Model Technical Answer
How does SpotSync prevent two users from booking the same slot at the same time?	We utilize SQLite atomic transactions with `BEGIN IMMEDIATE` locking. When a reservation request arrives, the slot's current status and active reservation records are checked inside the locked transaction. If another user commits first, the second transaction is immediately rolled back and returns an HTTP 409 Conflict error.
How do you achieve real-time availability updates across clients?	Instead of client-side polling, the backend maintains a WebSocket connection pool (`/ws/lots`). Whenever a slot changes state (booked, cancelled, entered, or maintenance), the server broadcasts a lightweight JSON payload to all active clients within 100 milliseconds, fulfilling REQ-8 (<5 seconds).
How are QR codes protected against forgery and reuse?	Each QR token embeds the reservation ID, user ID, and slot ID signed with HMAC-SHA256 using a server secret key. Upon scanning, the server recalculates and verifies the cryptographic signature, verifies that the ticket is in the correct state, and marks it as consumed to prevent replay attacks.
How does the system calculate parking fees upon exit?	Upon entry scan, the server records the exact UTC entry timestamp. Upon exit scan, the departure timestamp is logged and the duration is computed in seconds. Hours are rounded up to billable increments multiplied by the lot's rate per hour, ensuring accurate pricing.
How is offline ticketing achieved?	Whenever an online reservation is confirmed, the ticket payload and cryptographic token are serialized and cached in browser `localStorage` . Even if the driver loses internet connection in an underground garage, the pass renders offline directly from local storage.

6. Conclusion

SpotSync successfully implements all functional and non-functional requirements established in the SRS v1.1 and RMMM Plan. By unifying real-time WebSocket communication, atomic database concurrency controls, cryptographic ticketing, and responsive map visualizations, the application demonstrates an end-to-end smart parking management platform ready for real-world deployment.